PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Construcción API REST con Node.js, Express y MySQL

Presentado a: Instructora Alexandra Muñoz

Por Aprendiz: **Nicolas Ruiz Colorado**

Ficha: 3312932

Competencia:

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **8** de **noviembre** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Construcción de una API REST con Node.js, Express y MySQL 4

1. Construcción de la Base de Datos 4

2. Configuración Inicial del Servidor 5

2.1. Inicialización del proyecto 5

2.2. Instalación de dependencias 6

2.3. Configuración del entorno y estructura del proyecto 7

2.4. Configuración express.json(), cors() 8

2.5. Ejecución del servidor 9

2.6. Análisis 9

3. Creación de API de prueba 10

3.1. Rutas de la API de Prueba 11

3.2. App.js de API de Prueba 11

3.3. Pruebas y Análisis 12

4. Creación de API Usuarios 15

4.1. Rutas Api Usuarios 17

4.2. App.js de API Usuarios 18

4.3. Pruebas y Análisis 19

5. Creación API productos..... 24

5.1. Rutas API productos..... 27

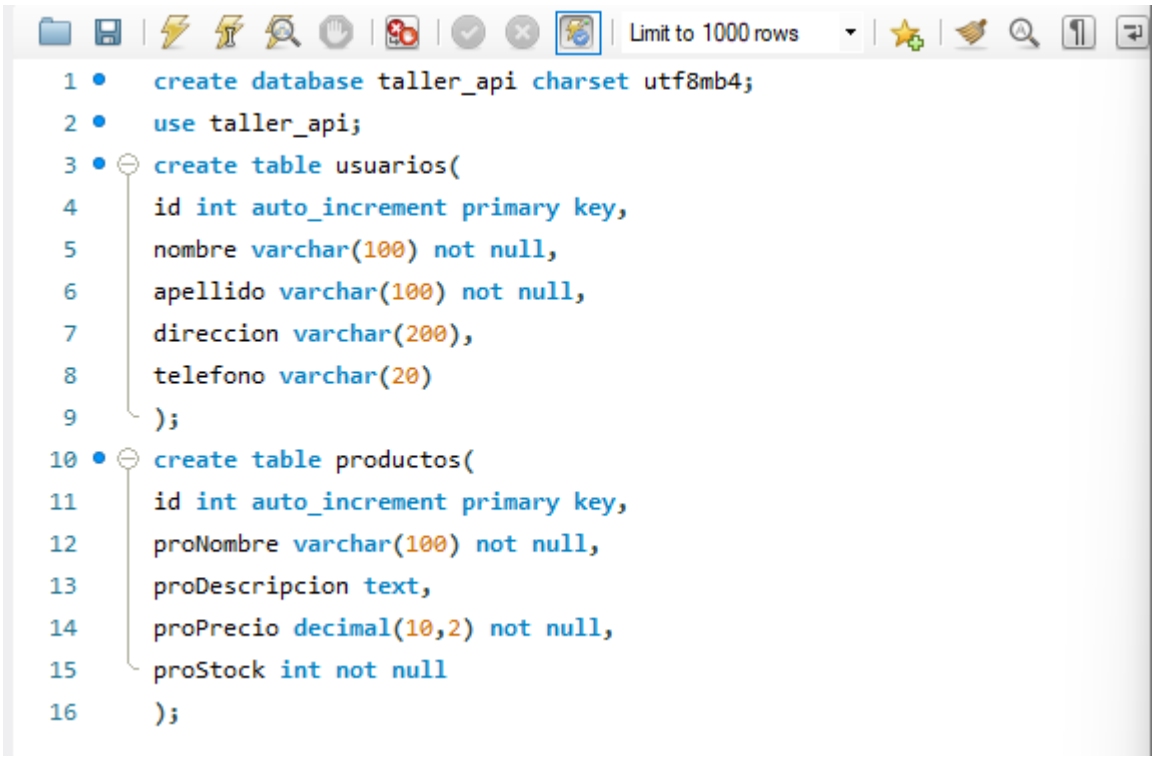
5.2. App.js API productos 28

5.3. Pruebas y Análisis 28

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Construcción de una API REST con Node.js, Express y MySQL

1. Construcción de la Base de Datos

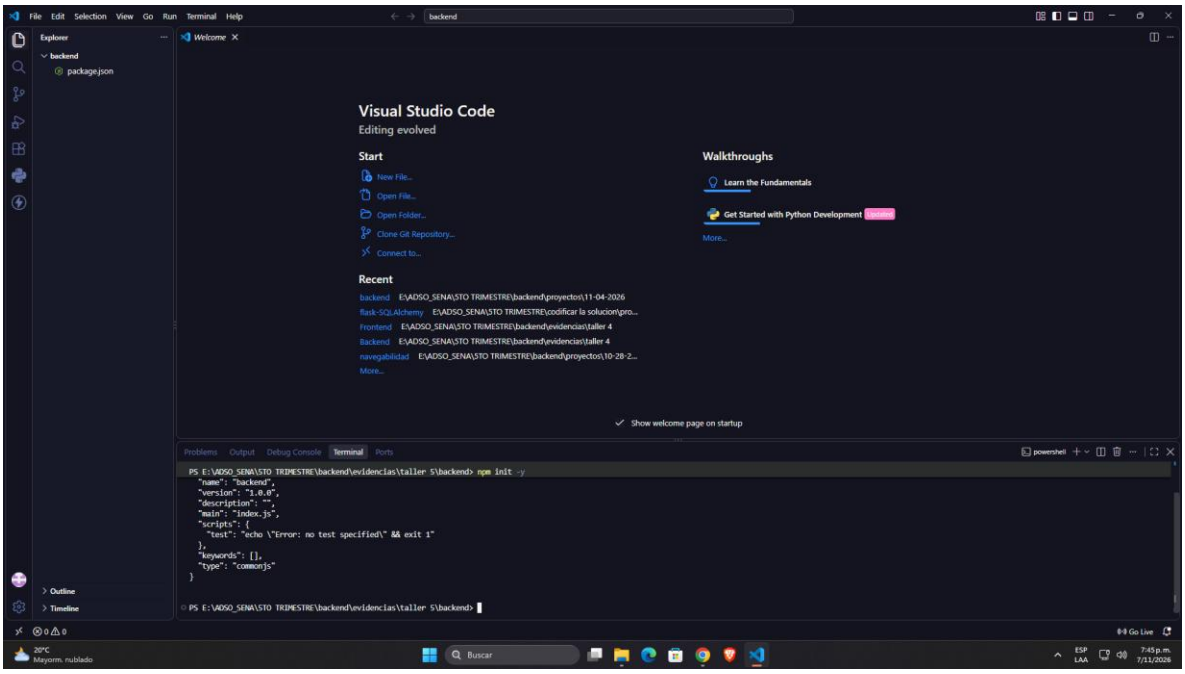



```sql
create database taller_api charset utf8mb4;
use taller_api;
create table usuarios(
id int auto_increment primary key,
nombre varchar(100) not null,
apellido varchar(100) not null,
direccion varchar(200),
telefono varchar(20)
);
create table productos(
id int auto_increment primary key,
proNombre varchar(100) not null,
proDescripcion text,
proPrecio decimal(10,2) not null,
proStock int not null
);
```

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

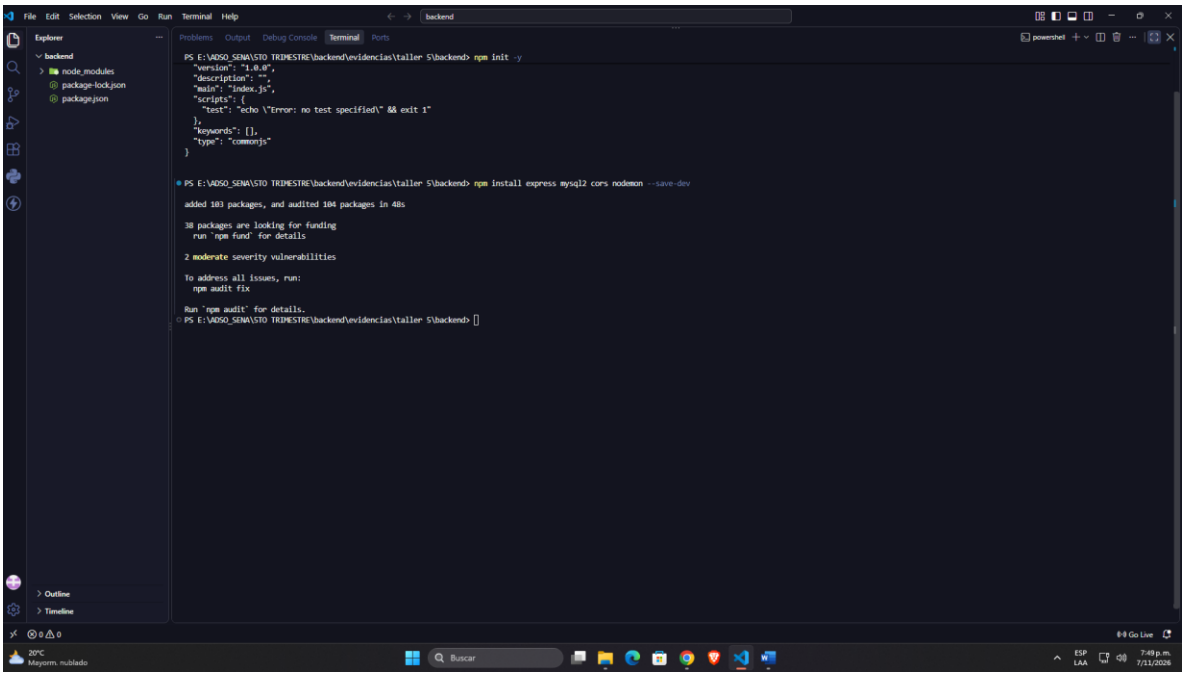
2. Configuración Inicial del Servidor

2.1. Inicialización del proyecto



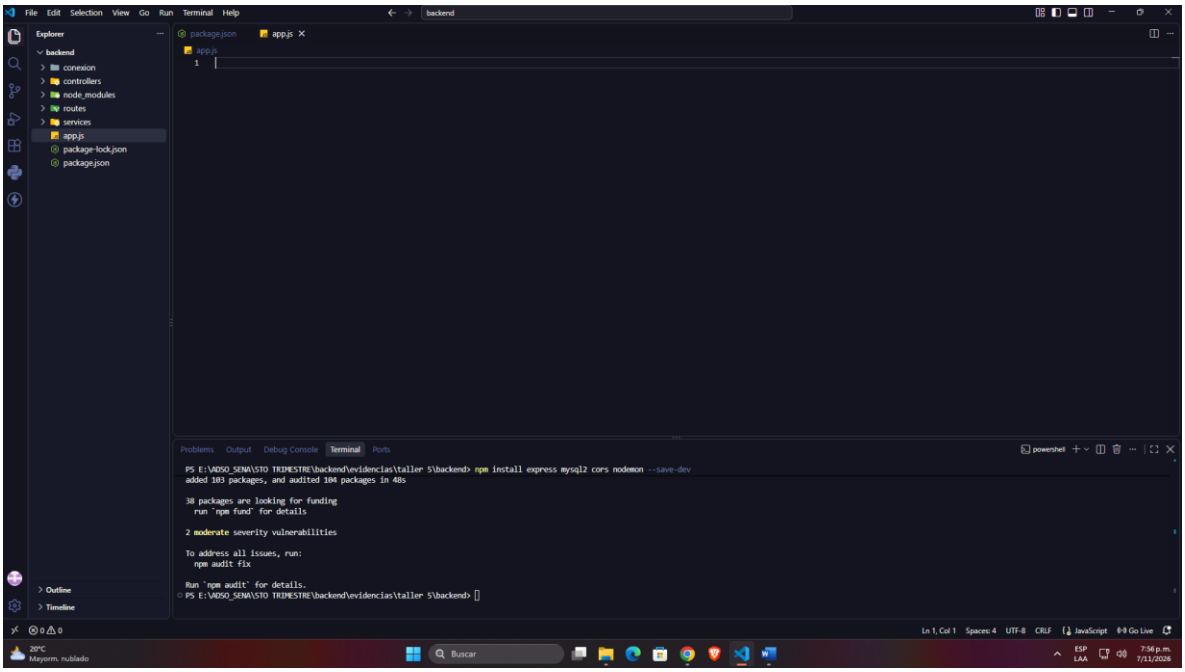
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2.2. Instalación de dependencias



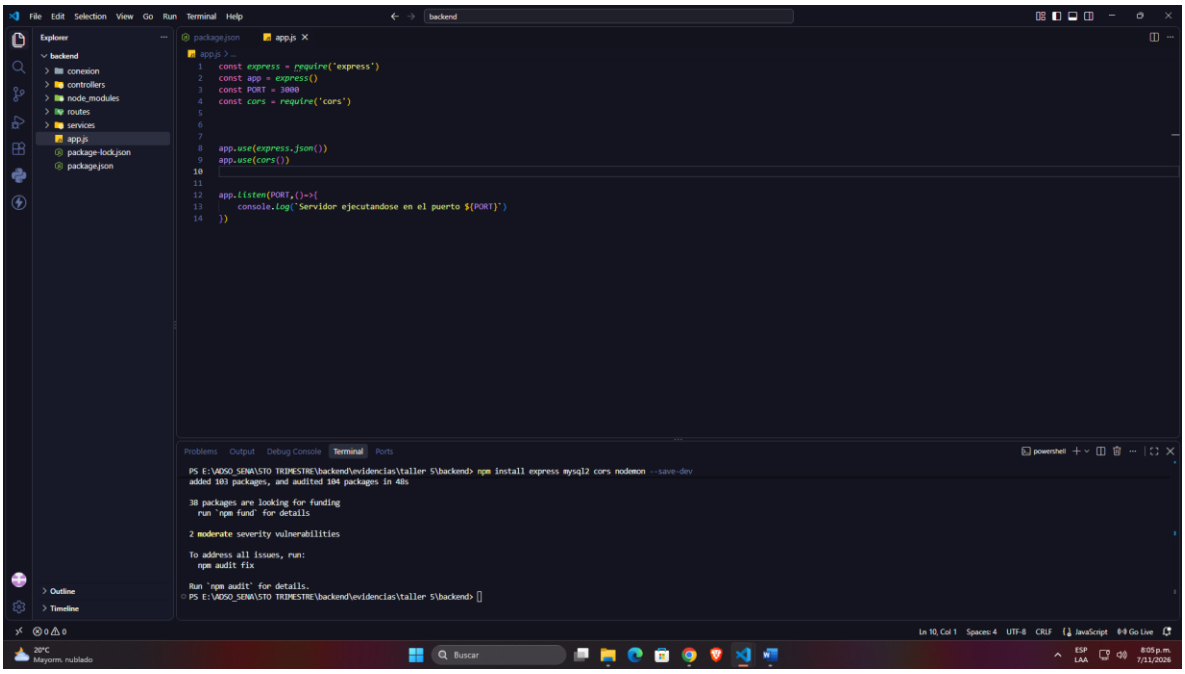
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2.3. Configuración del entorno y estructura del proyecto

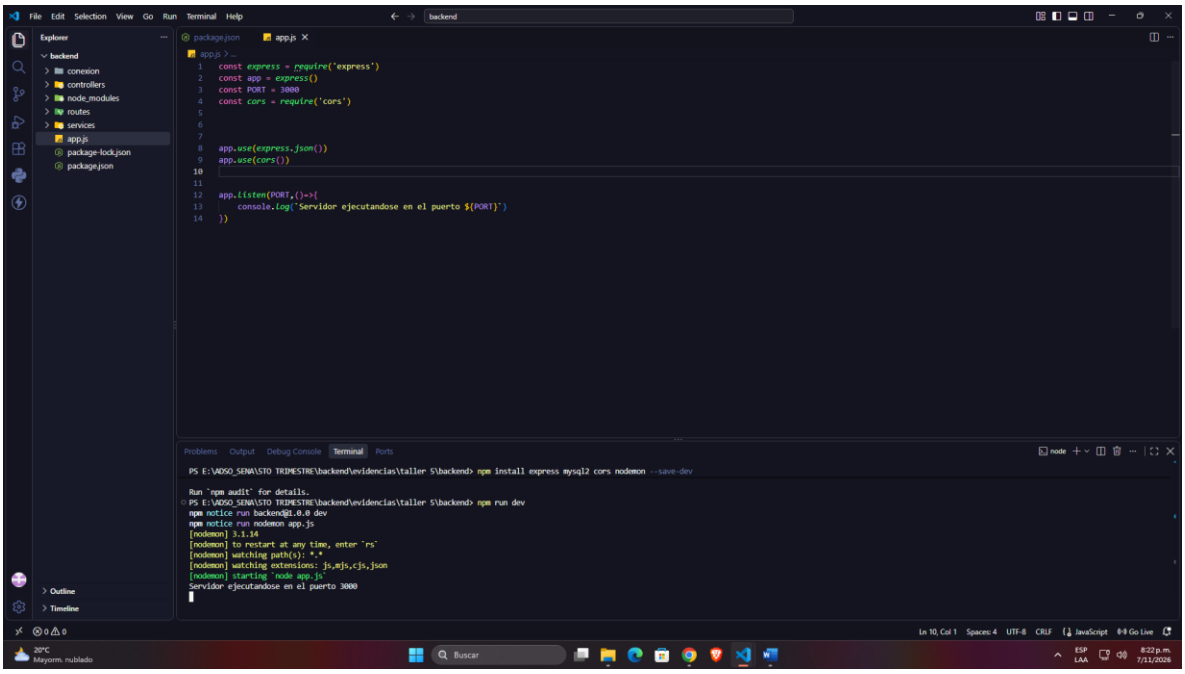


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

2.4. Configuración express.json(), cors()



2.5. Ejecución del servidor



2.6. Análisis

2.6.1. ¿Qué sucede si no instala Express?

Express es un framework de Node.js que permite crear servidores y desarrollar API, si no se instala esta dependencia o framework, tendría que utilizar funciones nativas de Node.js y el código será más complejo.

2.6.2. ¿Qué sucede si olvida ejecutar express?json() y se envía un cuerpo JSON?

Express.json() es un middleware de Express que permite interpretar los datos enviados en un formato JSON en el cuerpo de una solicitud, al no ejecutarlo el req.body tendría un valor undefined y no se podría utilizar la información suministrada en la petición

2.6.3. ¿Qué sucede si el puerto 3000 ya está ocupado?

Si el puerto esta siendo utilizado por otro programa o servidor la aplicación no se podría ejecutar

2.6.4. ¿Qué sucede si escribe incorrectamente la ruta de importación de un módulo?

Si se escribe incorrectamente la ruta de importación de un módulo Node.js no podría encontrar el archivo que se necesita utilizar y mostrará un error.

3. Creación de API de prueba

```
class PruebaController{
    bienvenida(req,res){
        res.json({'mensaje':'Bienvenido a mi APP'})
    }
    verificar(req,res){
        res.json({'mensaje':'Servidor funcionando
correctamente','estado':true})
    }
    recibir(req,res){
        const{nombre,edad}=req.body
        res.json({
            'mensaje':'Datos Recibidos correctamente',
            'nombre':nombre,
            'edad':edad
        })
    }
    sumar(req,res){
        const{numero1,numero2}=req.body
        res.json({
            resultado:numero1+numero2
        })
    }
}
module.exports=PruebaController
```

3.1. Rutas de la API de Prueba

```
let express = require('express')
let PruebaController = require('../controllers/PruebaController')
let rutaPrueba = express.Router()
let pruebaC= new PruebaController()
rutaPrueba.get('/bienvenida',pruebaC.bienvenida)
rutaPrueba.get('/verificar',pruebaC.verificar)
rutaPrueba.post('/recibir',pruebaC.recibir)
rutaPrueba.post('/sumar',pruebaC.sumar)

module.exports=rutaPrueba
```

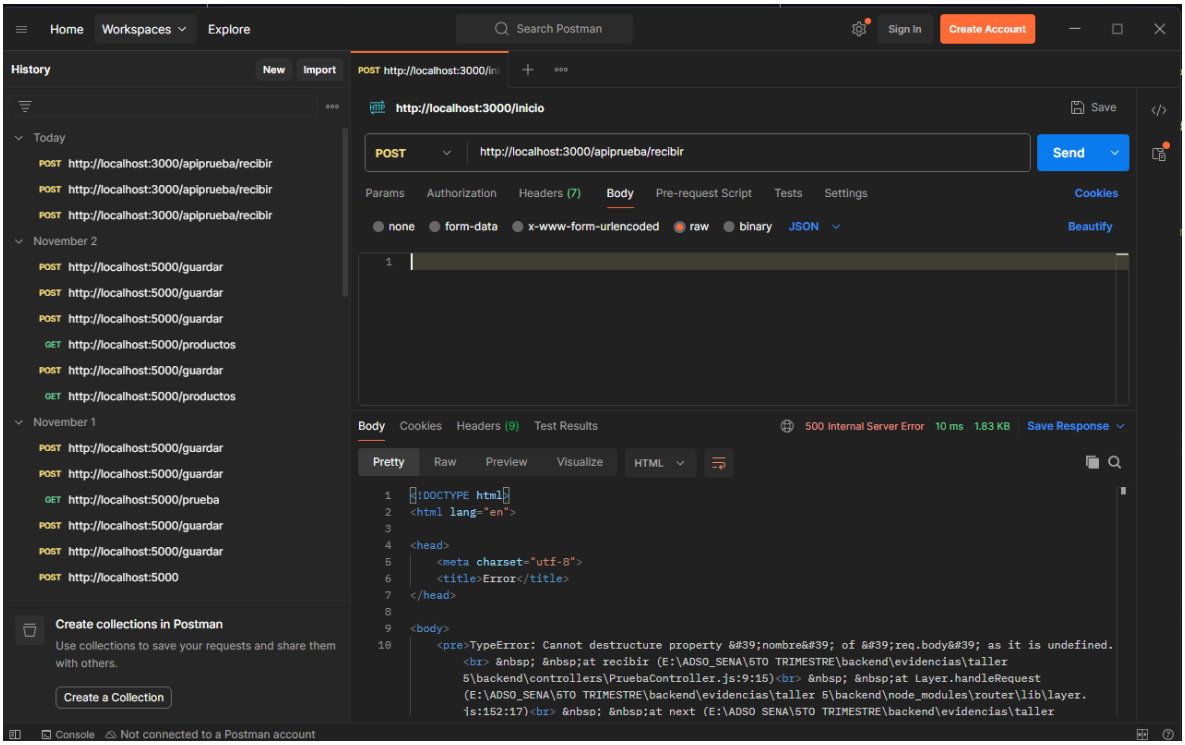
3.2. App.js de API de Prueba

```
app.use('/apiprueba',rutaPrueba)
```

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

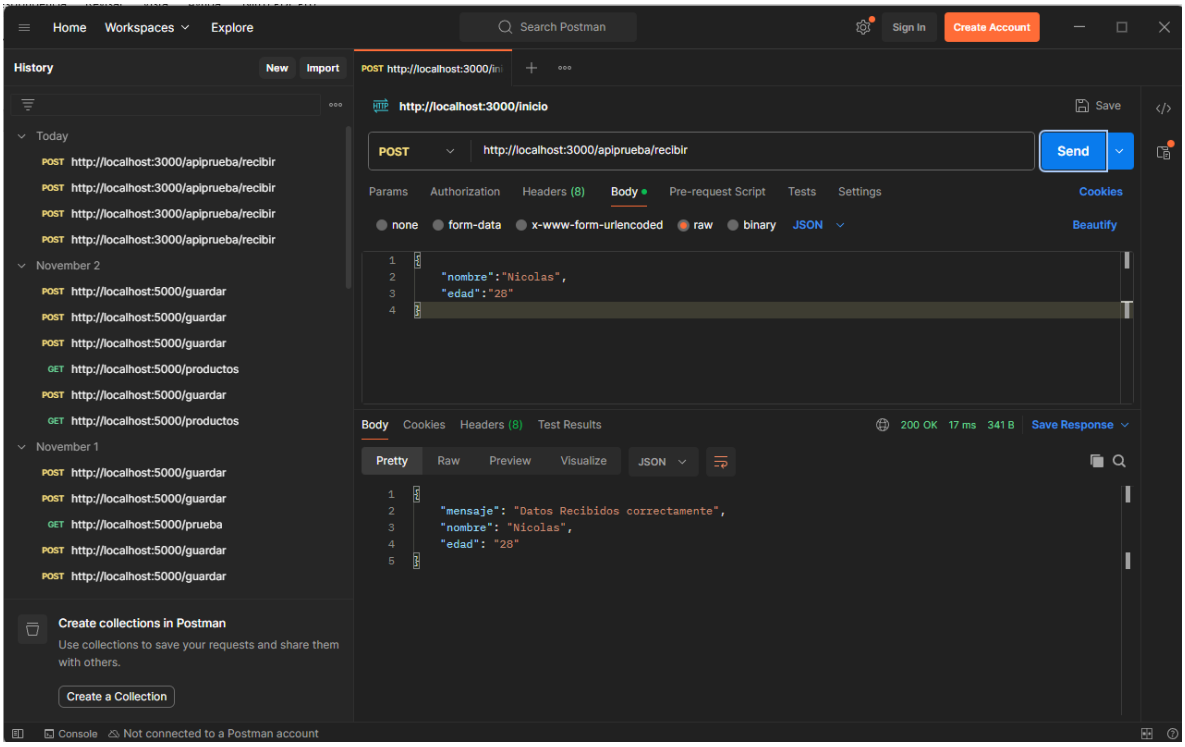
3.3. Pruebas y Análisis

3.3.1. Enviar una solicitud POST sin información



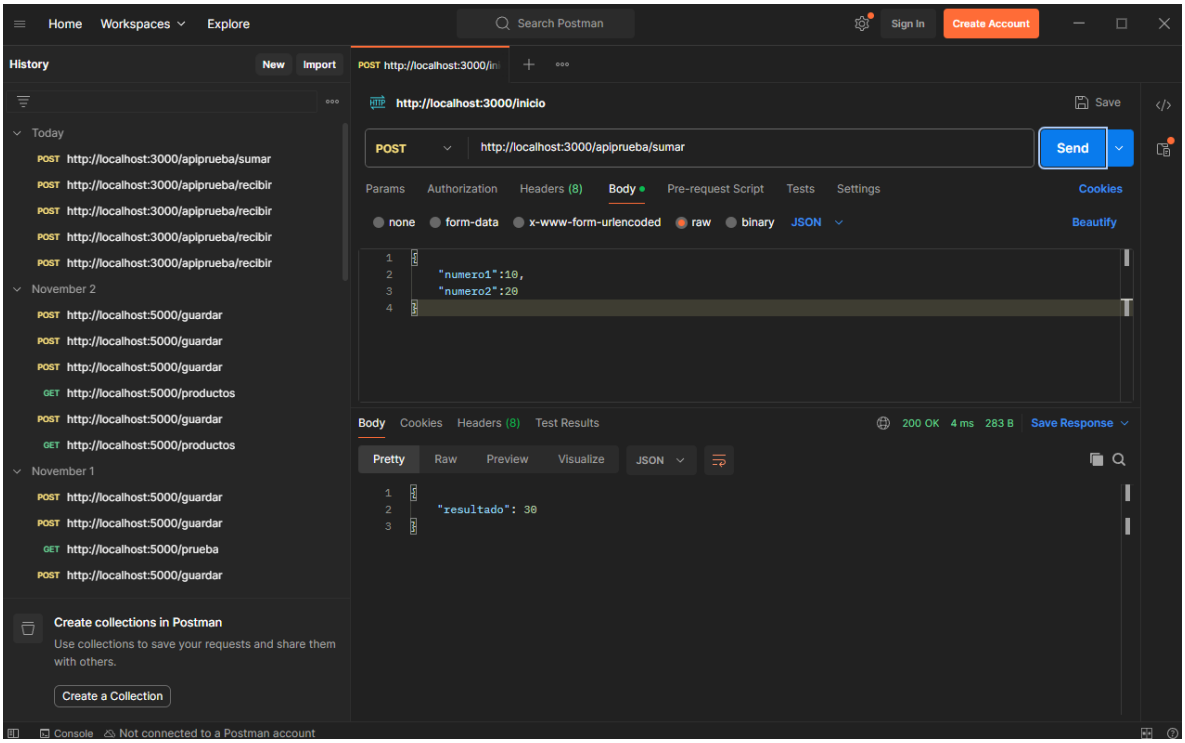
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

3.3.2. Enviar una edad como texto y explicar si el servidor la acepta

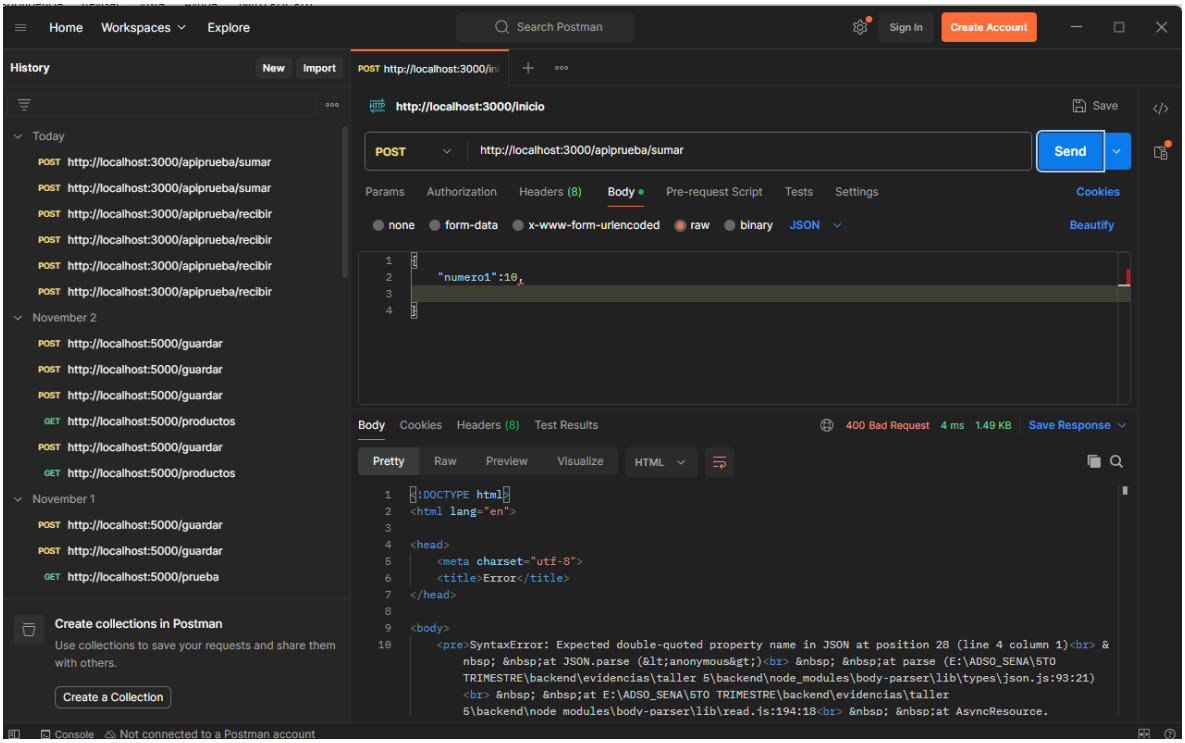


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

3.3.3. Enviar números en lugar de cadenas al método sumar



3.3.4. Omitir uno de los campos requeridos



3.3.5. Explicar que sucede al intentar leer req.body cuando req es undefined

Se produciría un error porque estoy intentando obtener una propiedad de una variable que no contiene ningún objeto.

4. Creación de API Usuarios

```
let conexion = require('../conexion/db')
class UsuarioController{
    guardarUsuario(req,res){
        const{nombre,apellido,direccion,telefono}=req.body
        conexion.query('INSERT INTO usuarios
(nombre,apellido,direccion,telefono) VALUES
(?,?,?,?)',[nombre,apellido,direccion,telefono],(error)=>{
            if(error){
                return res.json({
```

```
                    'mensaje':'Error al registrar el usuario'
                })
            }
            else{
                res.json({
                    'mensaje':'Usuario registrado correctamente'
                })
            }
        })
    }
    traerUsuario(req,res){
        conexion.query('SELECT * FROM usuarios',(error,resultado)=>{
            if(error){
                return res.json({
                    'mensaje':'Error al traer datos del usuario'
                })
            }
            else{
                return res.json(resultado)
            }
        })
    }
    buscarUsuario(req,res){
        const id =req.params.id
        conexion.query('SELECT * FROM usuarios WHERE
id=?',[id],(error,resultado)=>{
            if (error){
                return res.status(500).json({
                    'mensaje':'Error no se pudo encontrar un usario con
este id '
                })
            }
            else{
                return res.json(resultado)
            }
        })
    }
    actualizarUsuario(req,res){
        const id = req.params.id
        const {nombre,apellido,direccion,telefono}=req.body
```

```
        conexion.query('UPADTE usuarios SET nombre = ?, apellido = ?,
direccion = ?, telefono = ? WHERE id
=?',[nombre,apellido,direccion,telefono,id],(error,resultado)=>{
            if(error){
                res.status(500).json({
                    'mensaje':'No se pudo actualizar el usuario'
                })
            }
            else{
                res.json(resultado)
            }
        })
    }
    eliminarUsuario(req,res){
        const id = req.params.id
        conexion.query('DELETE from usuarios WHERE id =
?',[id],(error,resultado)=>{
            if(error){
                return res.status(500).json({
                    'mensaje':'Error al eliminar usuario'
                })
            }
            if(resultado.affectedRows===0){
                return res.json(404).json({
                    'mensaje':'Usuario no encontrado'
                })
            }
            res.json({
                'mensaje':'Usuario eliminado correctamente'
            })
        })
    }
}
module.exports = UsuarioController
```

4.1. Rutas Api Usuarios

```
let express = require('express')
```

```
let UsuarioController = require('../controllers/UsuarioController')

let rutaUsuarios = express.Router()
let usuario = new UsuarioController()
rutaUsuarios.post('/guardaruser',usuario.guardarUsuario)
rutaUsuarios.get('/traeruser',usuario.traerUsuario)
rutaUsuarios.get('/buscaruser',usuario.buscarUsuario)
rutaUsuarios.put('/actualizaruser',usuario.actualizarUsuario)
rutaUsuarios.delete('/eliminaruser',usuario.eliminarUsuario)

module.exports = rutaUsuarios
```

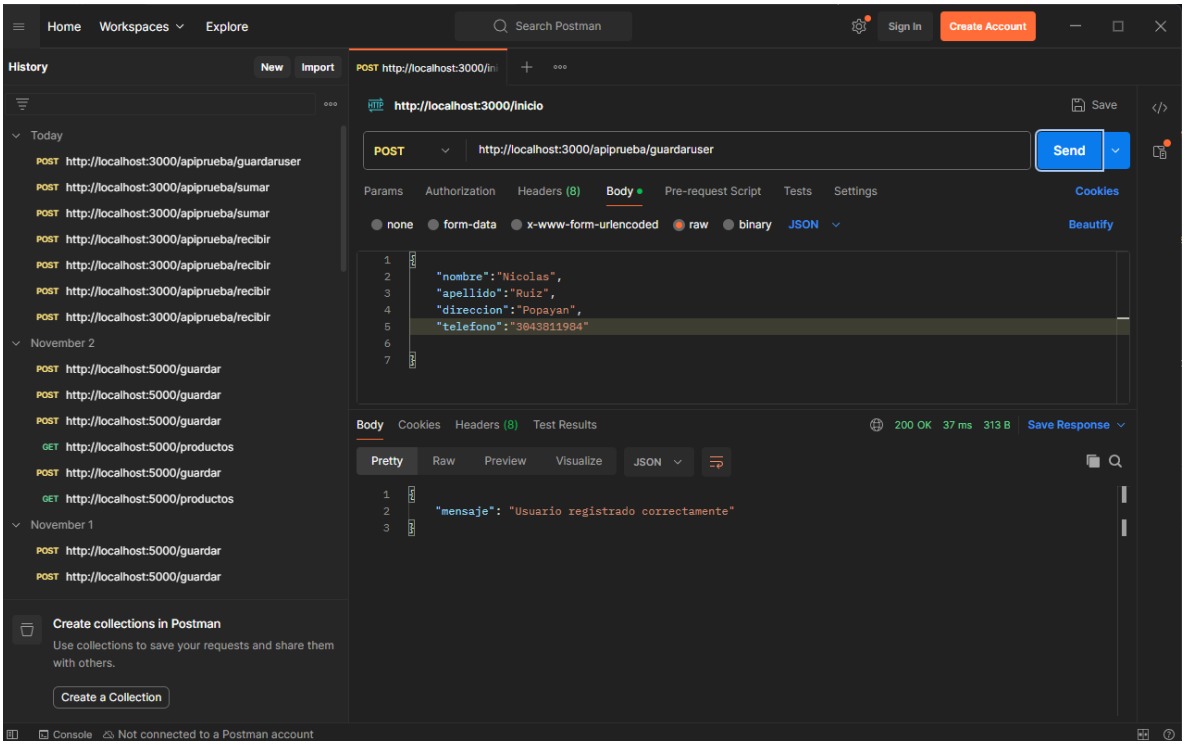
4.2. App.js de API Usuarios

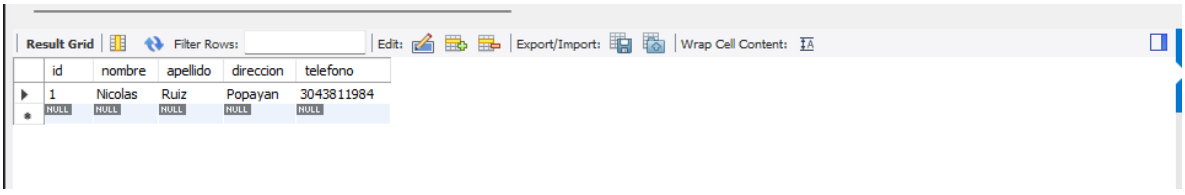
```
const rutaUsuarios = require('./routes/UsuarioRoutes')
app.use('/apiprueba',rutaUsuarios)
```

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

4.3. Pruebas y Análisis

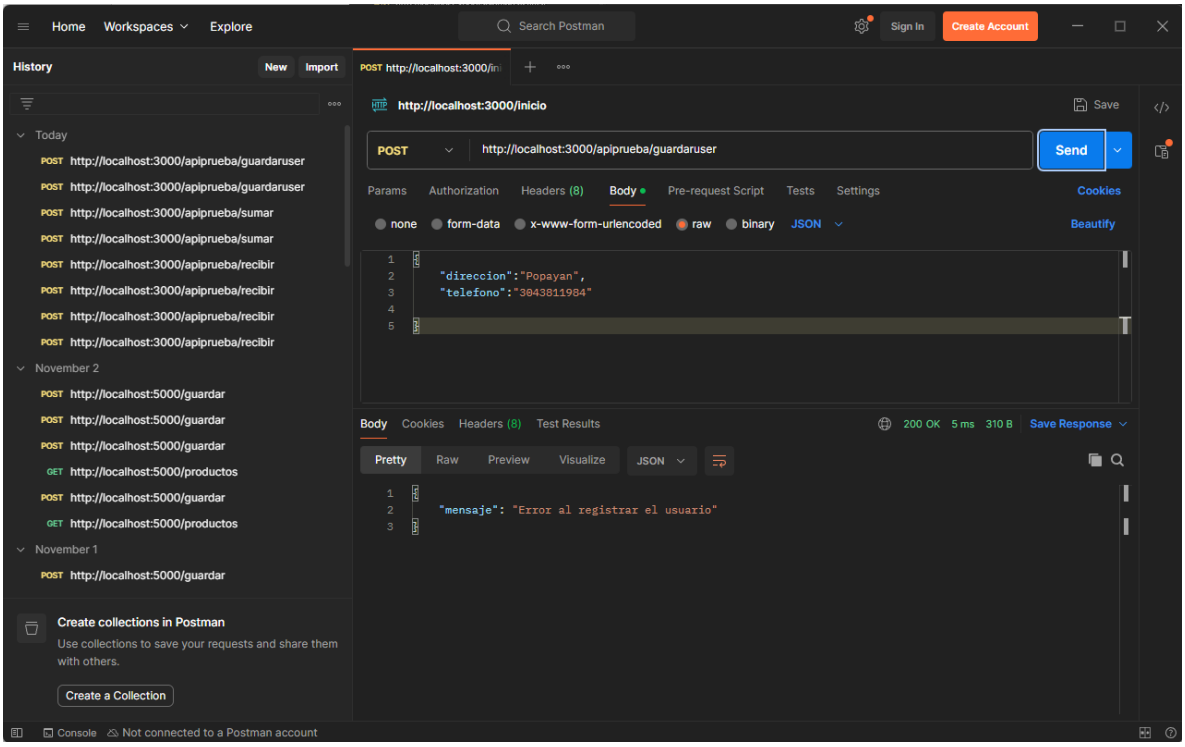
4.3.1. Registrar un usuario con todos los campos





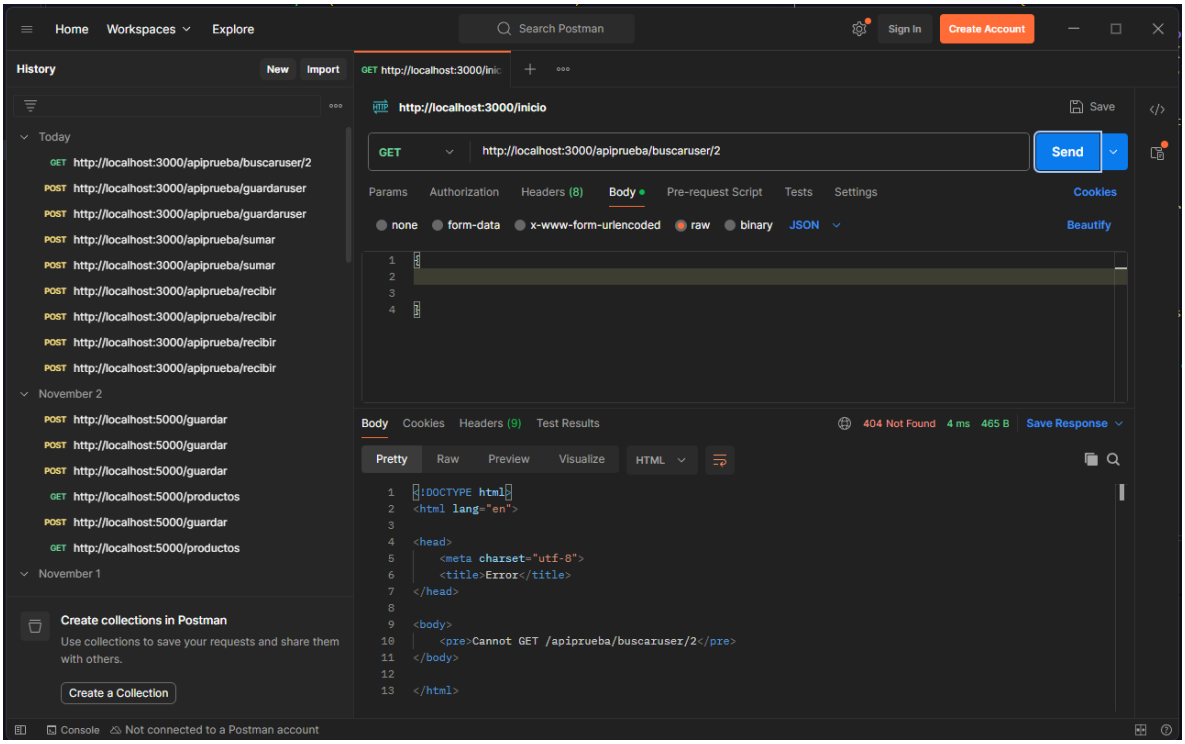
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

4.3.2. Intentar registrar un usuario sin nombre y apellido



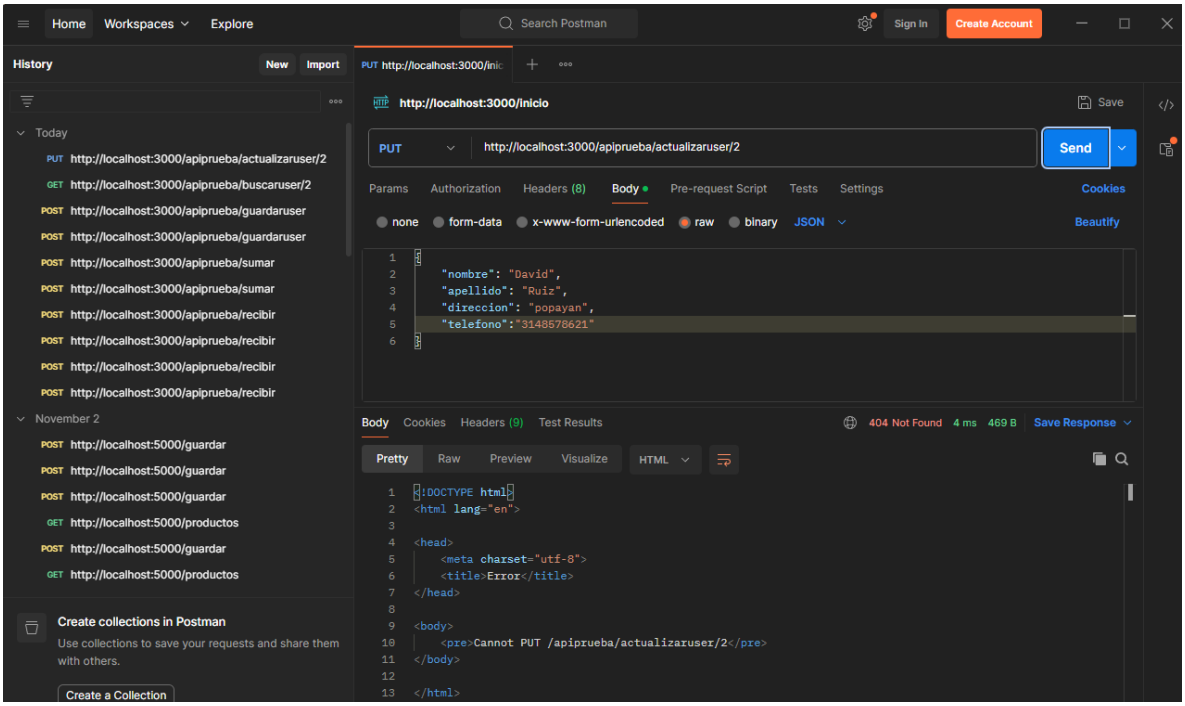
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

4.3.3. Consultar un identificador que no existe



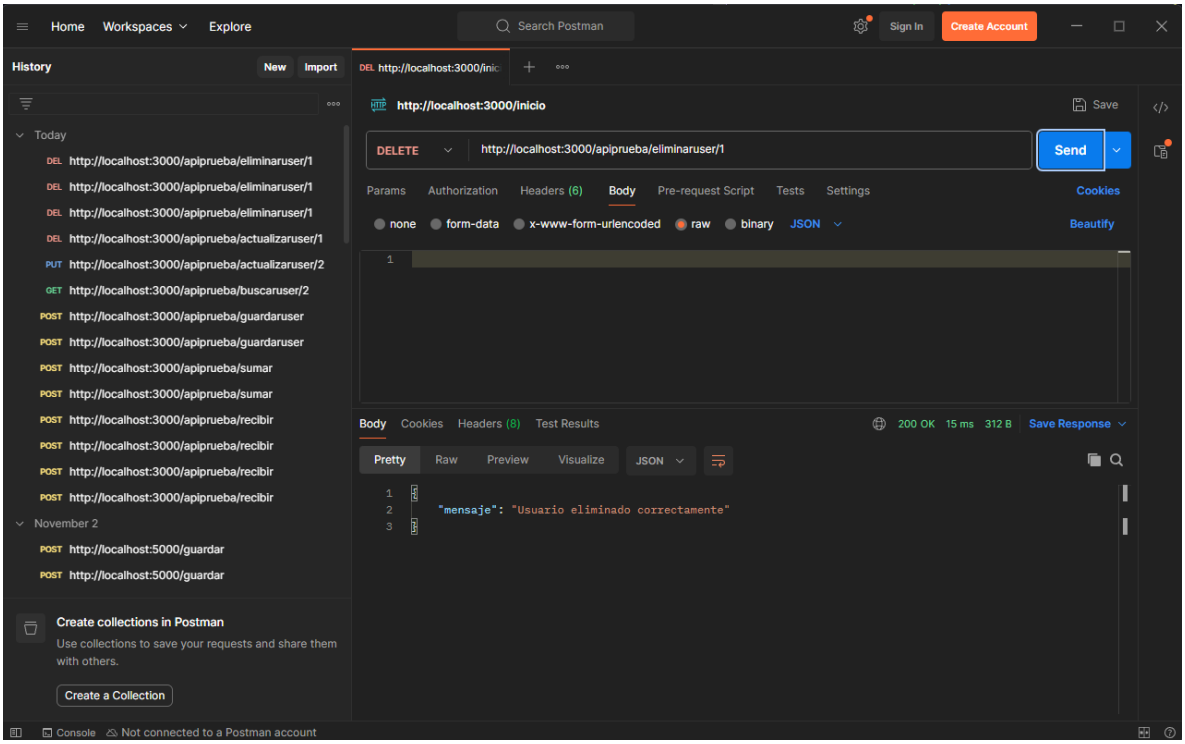
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

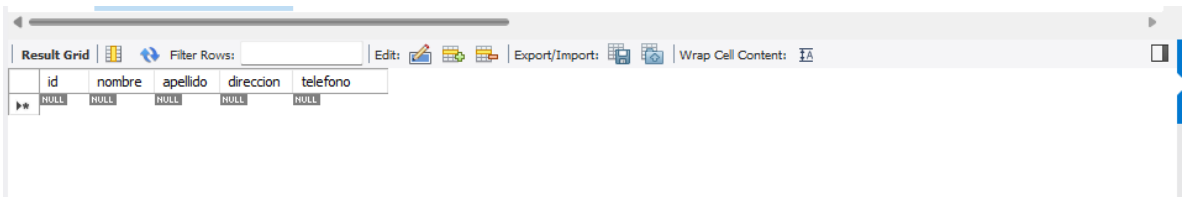
4.3.4. Actualizar un identificador inexistente



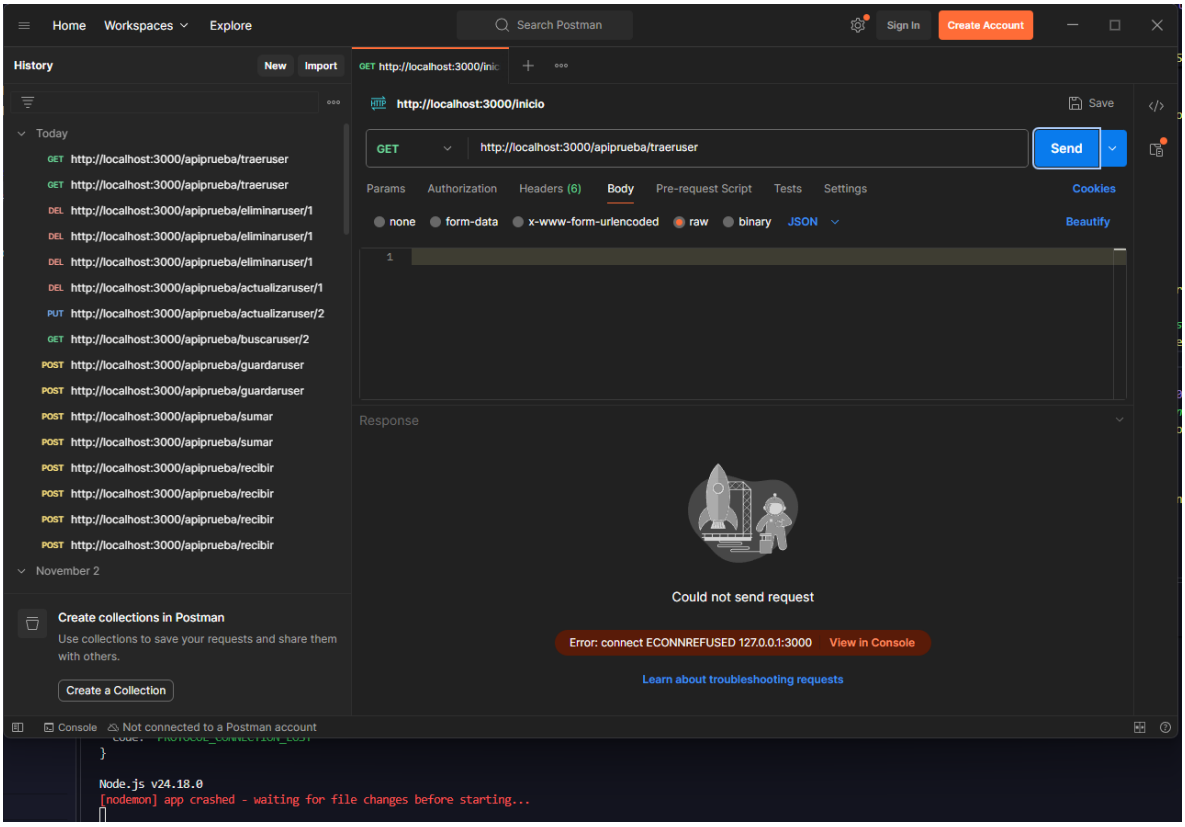
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

4.3.5. Eliminar un usuario y volver a consultar el mismo identificador





4.3.6. Desconectar MySQL e intentar consultar usuarios



5. Creación API productos

```
let conexion = require('../conexion/db')
class ProductosController{
    guardarProducto(req,res){
        const {proNombre,proDescripcion,proPrecio,proStock} = req.body
        conexion.query('INSERT INTO productos
(proNombre,proDescripcion,proPrecio,proStock) VALUES (?,?,?,?)',
[proNombre,proDescripcion,proPrecio,proStock],(error)=>{
            if(error){
                return res.json({
                    'mensaje':'Error al registrar el producto'
                })
            }
            else{
```

```
                return res.json({
                    'mensaje':'Producto registrado correctamente'
                })
            }
        })
    }
    traerProductos(req,res){
        conexion.query('SELECT * FROM productos',(error,resultado)=>{
            if(error){
                return res.json({
                    'mensaje':'Error al traer los datos de producto'
                })
            }
            else{
                return res.json(resultado)
            }
        })
    }
    buscarProducto(req,res){
        const id = req.params.id
        conexion.query('SELECT * FROM productos WHERE id =
?',[id],(error,resultado)=>{
            if(error){
                return res.json({
                    'mensaje':'Error no hay productos con este numero de
id'
                })
            }
            else{
                return res.json(resultado)
            }
        })
    }
    actualizarProducto(req,res){
        const id = req.params.id
        const {proNombre,proDescripcion,proPrecio,proStock}= req.body
        conexion.query('UPDATE productos SET proNombre = ?, proDescripcion
= ?, proPrecio = ?, proStock = ? WHERE id =
?',[proNombre,proDescripcion,proPrecio,proStock,id],(error,resultado)=>{
            if(error){
```

```
                return res.json({
                    'mensaje':'Error al actualizar el producto'
                })
            }
            else{
                return res.json(resultado)
            }
        })
    }
    eliminarProducto(req,res){
        const id = req.params.id
        conexion.query('DELETE from productos WHERE id =
?',[id],(error,resultado)=>{
            if(error){
                return res.status(500).json({
                    'mensaje':'Error al eliminar producto'
                })
            }
            if(resultado.affectedRows===0){
                return res.status(404).json({
                    'mensaje':'Producto no encontrado'
                })
            }
            res.json({
                'mensaje':'Producto eliminado correctamente'
            })
        })
    }
    productosAgotados(req,res){
        conexion.query('SELECT * FROM productos WHERE proStock =
0',(error,resultado)=>{
            if(error){
                return res.json({
                    'mensaje':'No hay productos sin stock'
                })
            }
            else{
                res.json(resultado)
            }
        })
```

```
    }
    bajoStock(req,res){
        conexion.query('SELECT * FROM productos WHERE proStock <
5',(error,resultado)=>{
            if(error){
                return res.json({
                    'mensaje':'No hay productos con bajo stock'
                })
            }
            else{
                res.json(resultado)
            }
        })
    }
}
module.exports = ProductosController
```

5.1. Rutas API productos

```
let express = require('express')
let ProductosController = require('../controllers/ProductosController')

let rutaProductos = express.Router()
let producto = new ProductosController()
rutaProductos.post('/guardarp',producto.guardarProducto)
rutaProductos.get('/traerp',producto.traerProductos)
rutaProductos.get('/buscarp',producto.buscarProducto)
rutaProductos.put('/actualizarp/:id',producto.actualizarProducto)
rutaProductos.delete('/eliminarp/:id',producto.eliminarProducto)
rutaProductos.get('/productosa',producto.productosAgotados)
rutaProductos.get('/bajostock',producto.bajoStock)

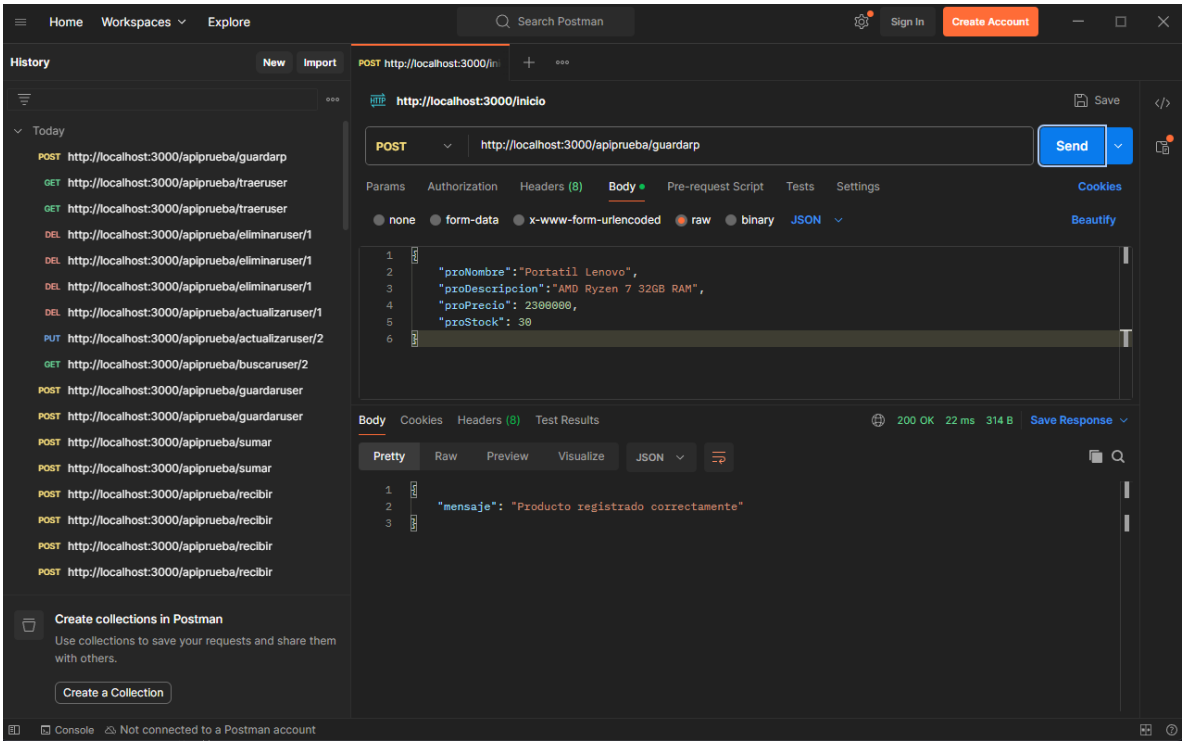
module.exports = rutaProductos
```

5.2. App.js API productos

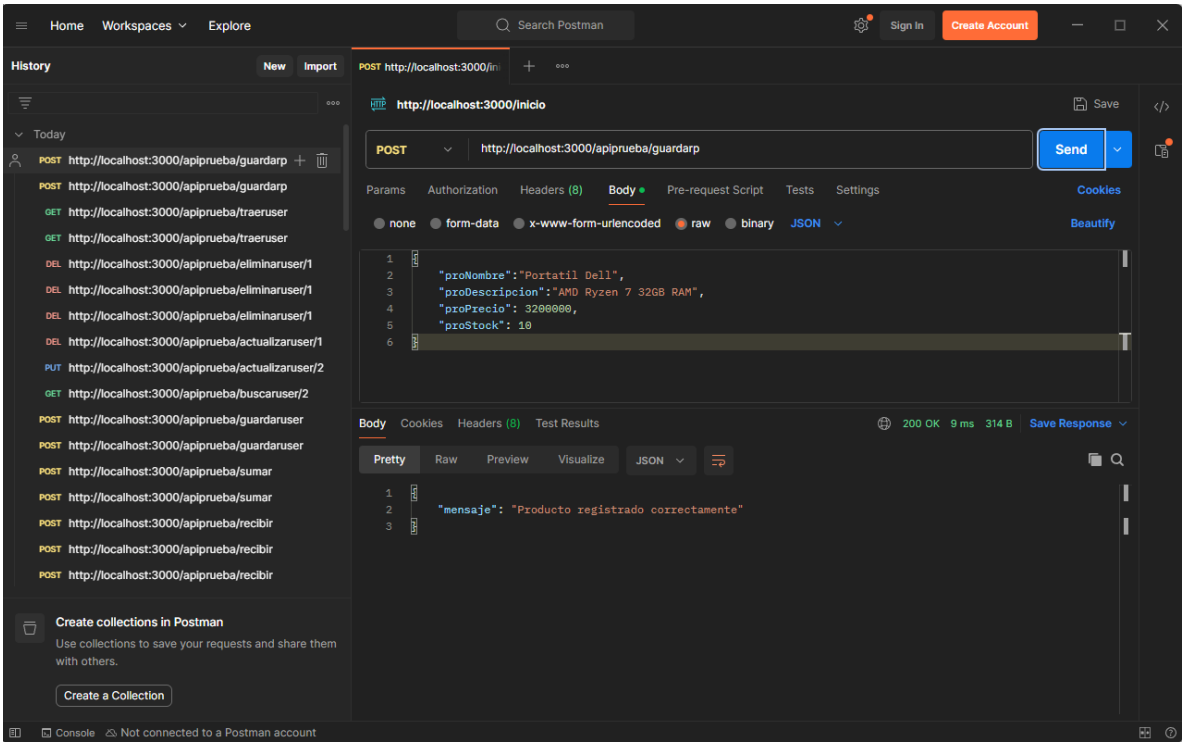
```
const rutaProductos = require('./routes/ProductoRoutes')
app.use('/apiprueba',rutaProductos)
```

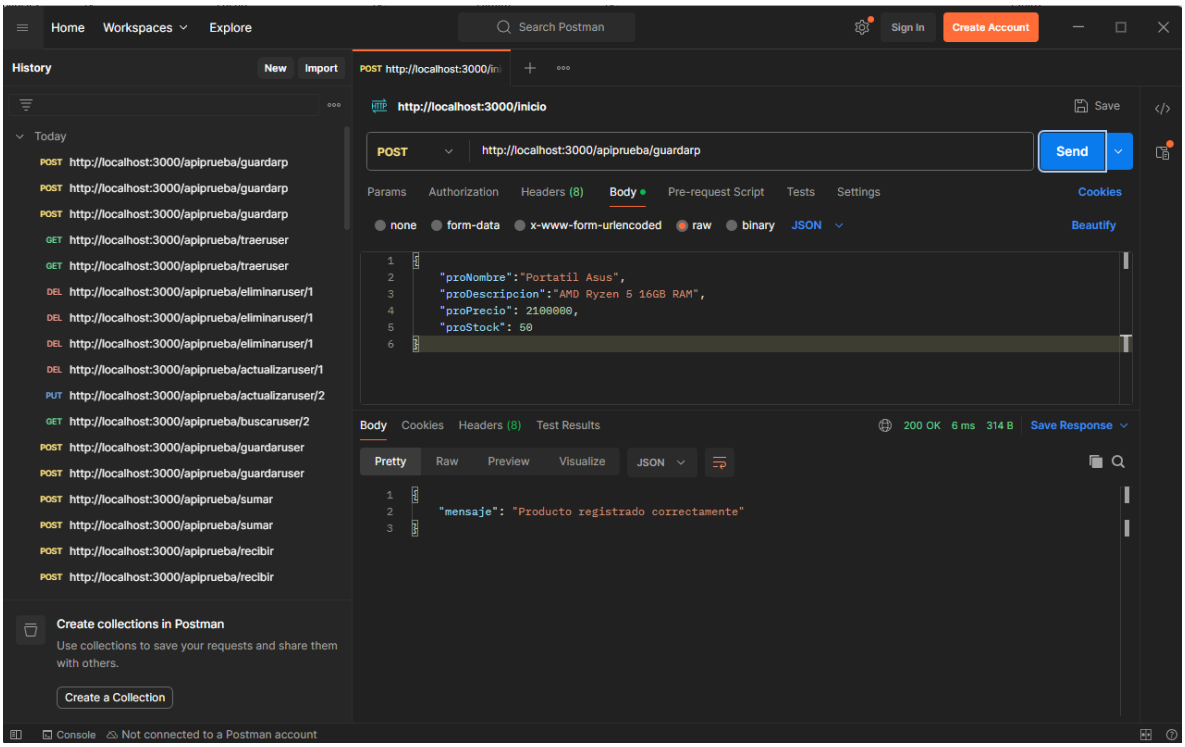
5.3. Pruebas y Análisis

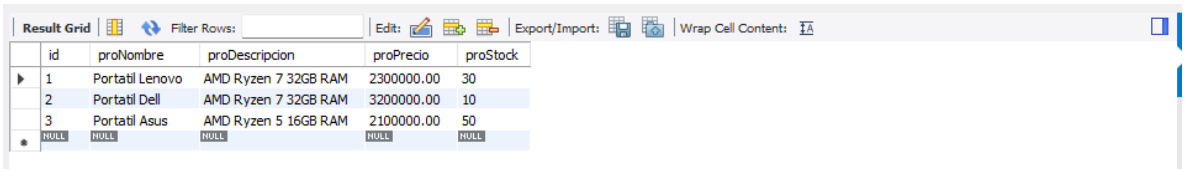
5.3.1. Registrar 3 productos con diferentes precios y cantidades



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



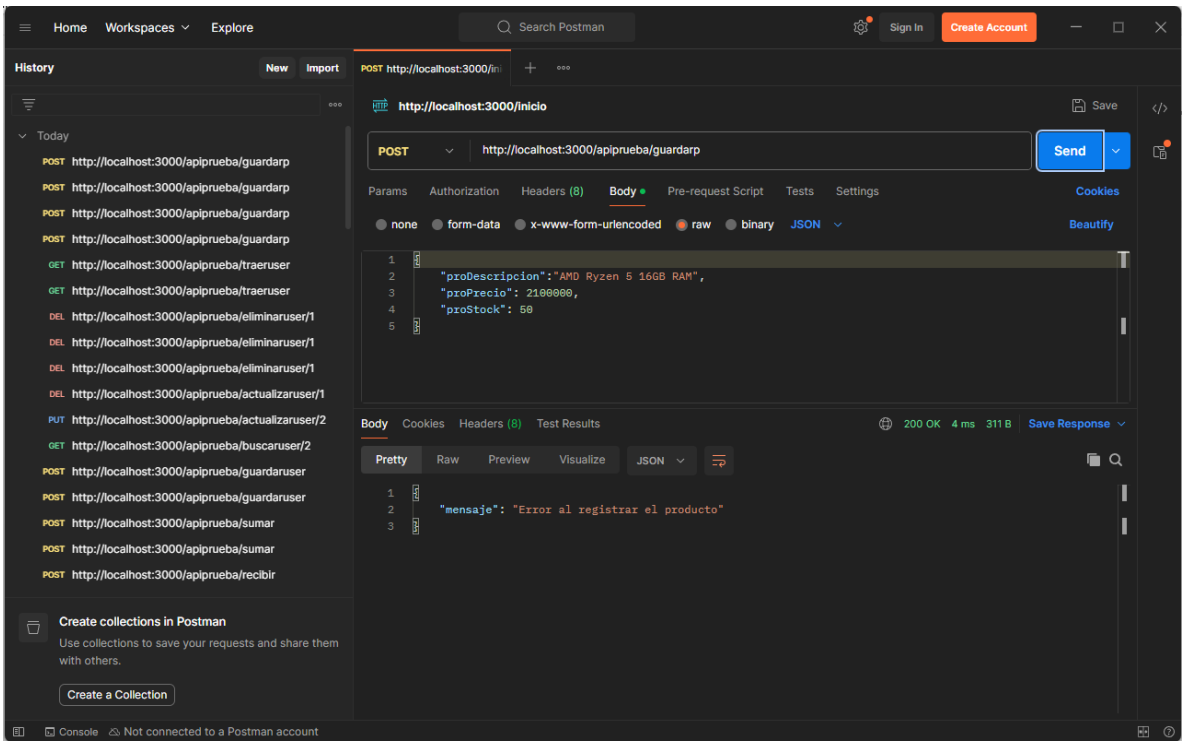




Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

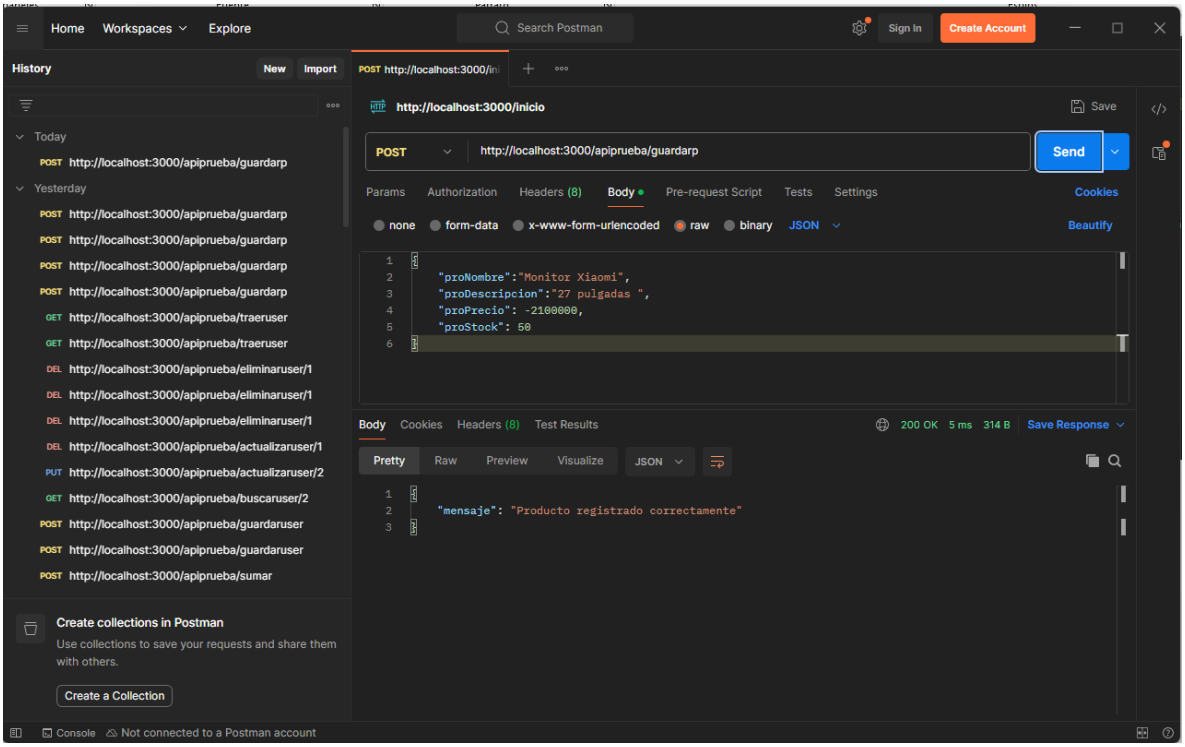
id	proNombre	proDescripcion	proPrecio	proStock
1	Portatil Lenovo	AMD Ryzen 7 32GB RAM	2300000.00	30
2	Portatil Dell	AMD Ryzen 7 32GB RAM	3200000.00	10
3	Portatil Asus	AMD Ryzen 5 16GB RAM	2100000.00	50
NULL	NULL	NULL	NULL	NULL

5.3.2. Intentar registrar un producto sin nombre



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

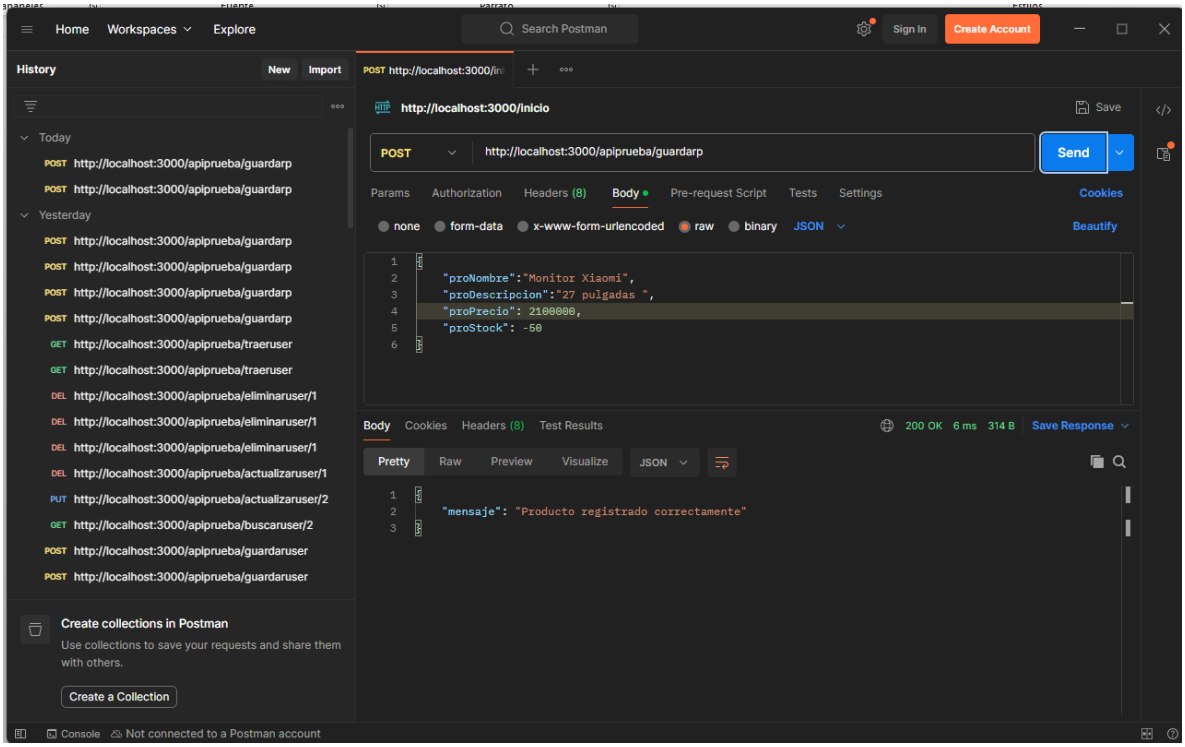
5.3.3. Enviar precio negativo y explicar cómo debería validarse

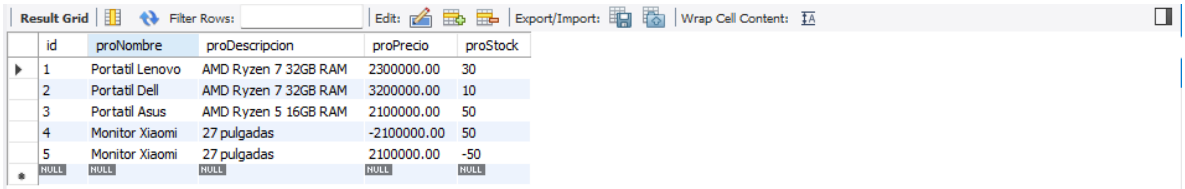


Se registra el producto porque no hay una validación del precio para que los valores no sean negativos

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

5.3.4. Enviar un stock negativo y explicar cómo debería validarse



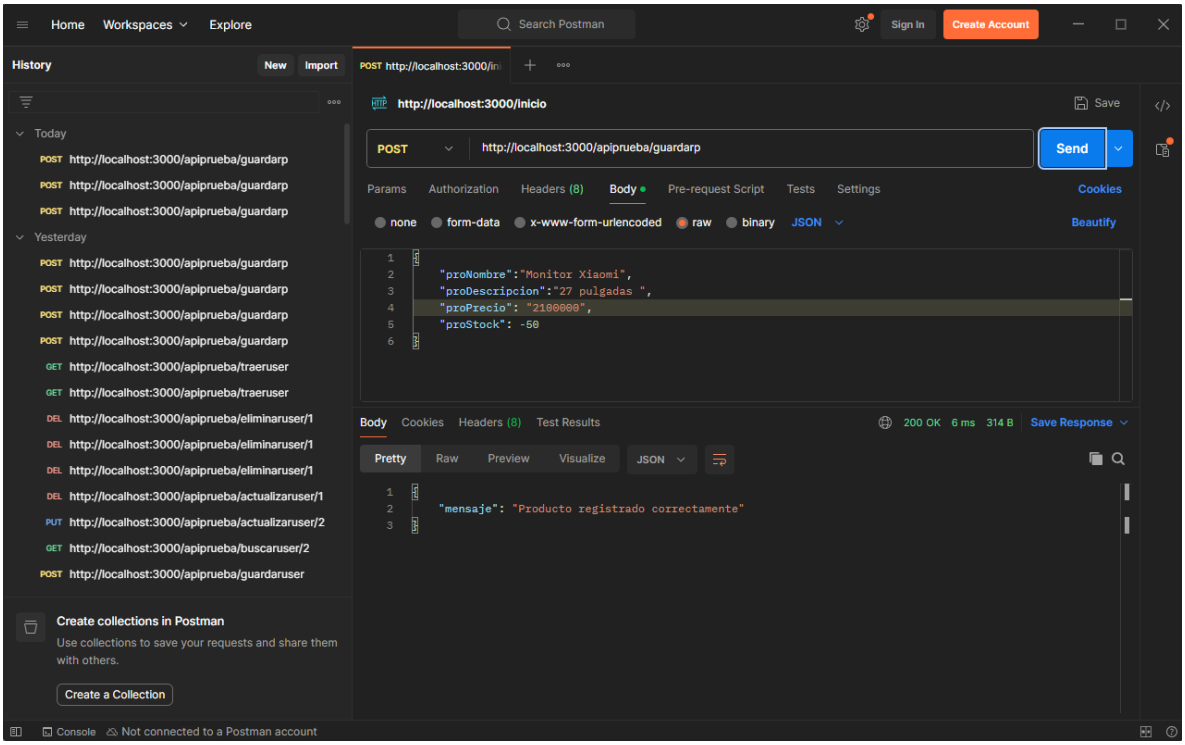


Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	proNombre	proDescripcion	proPrecio	proStock
1	Portatil Lenovo	AMD Ryzen 7 32GB RAM	2300000.00	30
2	Portatil Dell	AMD Ryzen 7 32GB RAM	3200000.00	10
3	Portatil Asus	AMD Ryzen 5 16GB RAM	2100000.00	50
4	Monitor Xiaomi	27 pulgadas	-2100000.00	50
5	Monitor Xiaomi	27 pulgadas	2100000.00	-50
NULL	NULL	NULL	NULL	NULL

Como en el ejemplo anterior igual recibe un stock o valor negativo. Ya que no hay validaciones definidas para comprobar si era o no negativo

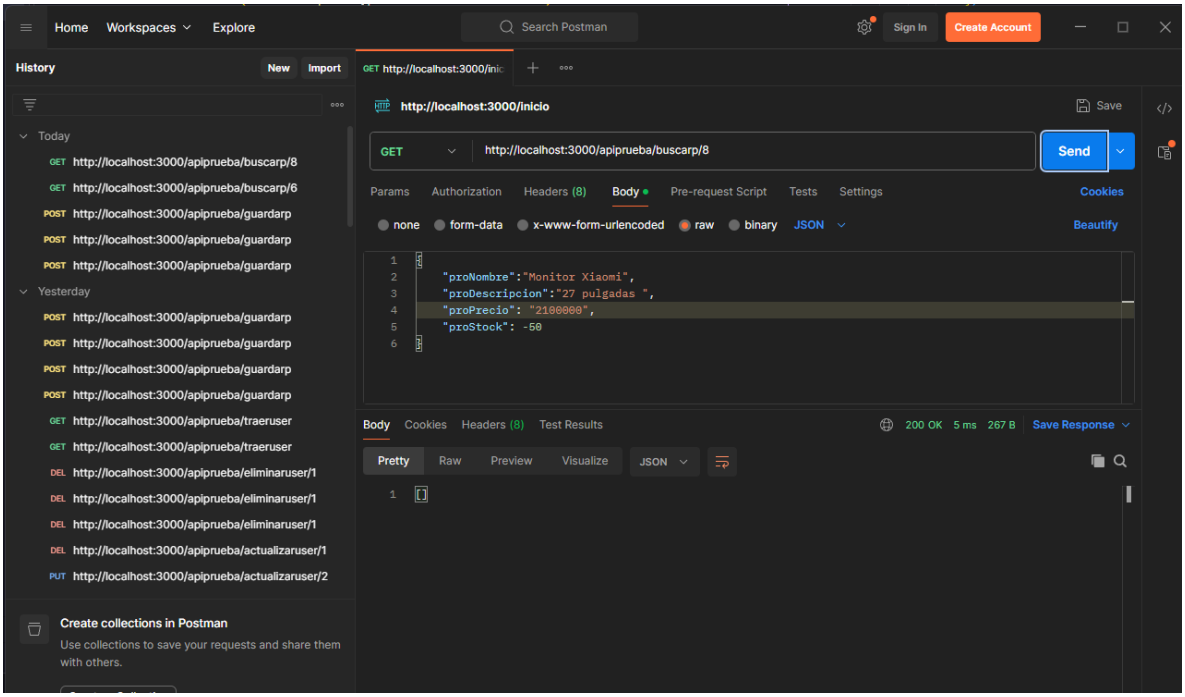
5.3.5. Enviar el precio como texto no numérico



Como no existe una regla de validaciones es decir una comprobación ni de tipo de dato, ni de valores; por lo cual se registra como correcto

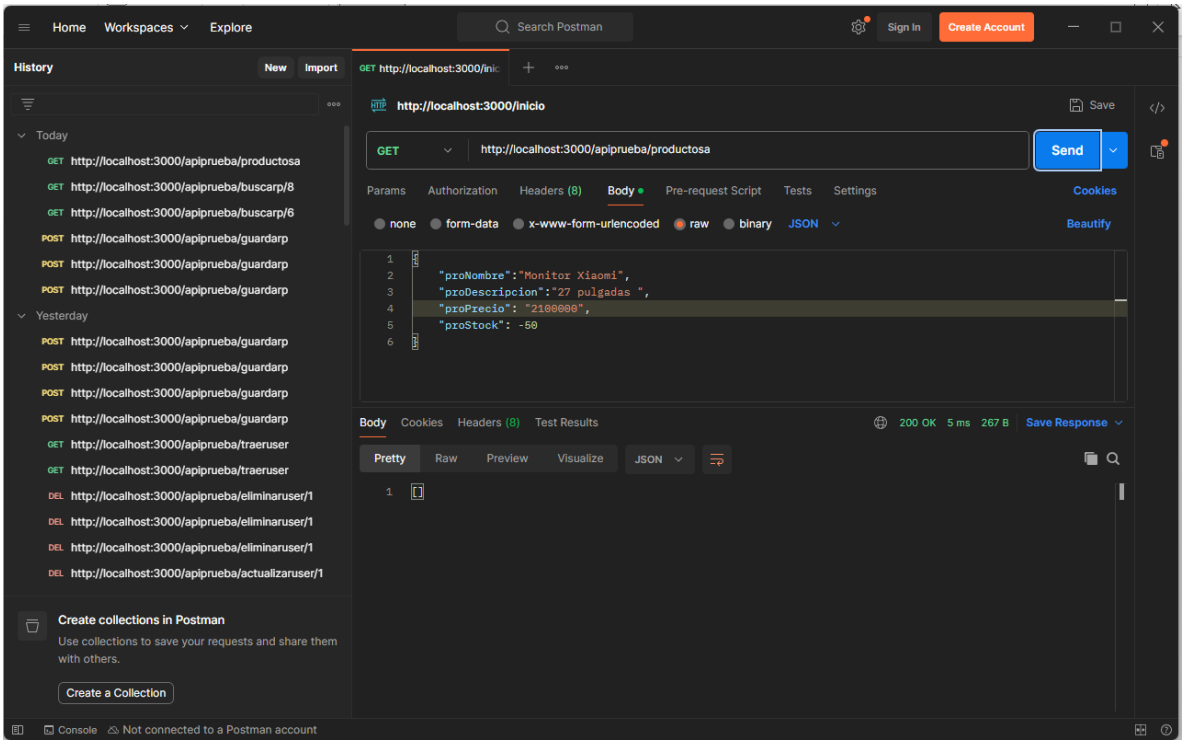
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

5.3.6. Buscar un producto inexistente



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

5.3.7. Consultar productos agotados y productos con bajo stock



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

